

Sincronização de Processos via Primitivas: Barreira e Fila FIFO (First-In, First-Out)

Marcus Sebastião Adriano Rocha Neto – GRR20240710

¹Departamento de Informática – Universidade Federal do Paraná (UFPR)
Bacharelado em Informática Biomédica
Disciplina: CI1215 – Sistemas Operacionais
Prof. Dr. Wagner M. Nunan Zola
Curitiba – PR – Brasil

Resumo. *Este relatório descreve a especificação, decisões de design e implementação do primeiro trabalho prático da disciplina de Sistemas Operacionais (CI1215). O objetivo principal consiste em coordenar a execução concorrente de múltiplos processos em nível de sistema operacional utilizando primitivas clássicas de sincronização: uma Barreira de Sincronização e uma Fila de Espera FIFO (First-In, First-Out) para controle de acesso exclusivo a um recurso compartilhado.*

1. Introdução

O desafio central do trabalho reside na transição correta de um ambiente multi-thread (onde a memória é compartilhada nativamente) para um ambiente multiprocesso criado por meio da chamada de sistema `fork()`. Sob esta visão, o isolamento de memória imposto pelo kernel do Linux exige o mapeamento explícito de regiões compartilhadas na memória RAM e a correta reconfiguração dos semáforos POSIX para operação interprocessos.

2. Estruturas de Dados Utilizadas

Todas as estruturas compartilhadas são definidas em `shared.h`

2.1. Estrutura da Barreira

```
typedef struct
{
    sem_t mutex;
    sem_t turnstile;
    int count;
    int n;
} barrier_t;
```

2.1.1. Descrição da Estrutura

- `mutex`: Protege a variável `count` contra acessos simultâneos
- `turnstile`: Bloqueia processos até que todos cheguem
- `count`: Conta quantos processos chegaram na barreira
- `n`: Número total de processos esperados

2.2. Estrutura da Fila FIFO

```
typedef struct
{
    sem_t mutex_interno;
    sem_t sem_privado[MAX_PROCS];
    int fila[MAX_PROCS];
    int head;
    int tail;
    int recurso_ocupado;
} fifoq_t;
```

2.2.1. Descrição da Estrutura

- mutex: Protege a variável count contra acessos simultâneos
- sem_privado: Semáforo individual para cada processo
- fila: Armazena os processos em espera
- head: Início da fila
- tail: Final da fila
- recurso_ocupado: Indica se o recurso está sendo utilizado

2.3. Estrutura Compartilhada Principal

```
typedef struct
{
    barrier_t barreira;
    fifoq_t fila;
} shared_data_t;
```

Esta estrutura concentra todos os recursos compartilhados que serão acessados pelos processos filhos.

3. Implementação das Primitivas

3.1. Implementação da Barreira

```
void init_barr(barrier_t *barr, int n)
{
    barr->n = n;
    barr->count = 0;
    sem_init(&barr->mutex, 1, 1); // Compartilhado entre processos
    sem_init(&barr->turnstile, 1, 0); // Compartilhado entre processos
}

void process_barrier(barrier_t *barr)
{
    // Trava mutex e incrementa contador de chegada
    sem_wait(&barr->mutex);
    barr->count++;
}
```

```

// Último a chegar libera a todos
if (barr->count == barr->n)
{
    for (int i = 0; i < (barr->n - 1); i++)
        sem_post(&barr->turnstile);

    // Prepara para reuso
    barr->count = 0;
    sem_post(&barr->mutex);
}
else
{
    sem_post(&barr->mutex);
    sem_wait(&barr->turnstile);
}
}

```

3.1.1. Descrição

- `void init_barr(barrier_t *barr, int n)`: Inicializa a barreira, configurando todos campos da estrutura.
- `void process_barrier(barrier_t *barr)`: Função que implementa o uso da barreira.

3.1.2. Funcionamento

A função `process_barrier` representa a passagem de um processo pela barreira.

1. `sem_wait(&barr->mutex)`: Trava o mutex para evitar que dois processos alterem o `count` simultaneamente.
2. `barr->count++`: Registra a chegada do novo processo incrementando o contador.
3. `if (barr->count == barr->n)`: Verifica se o processo que chegou é o último aguardado pelos demais.
4. `barr->count = 0`: Caso o processo que acabou de chegar seja o último esperado, zera o contado de chegada para que a barreira possa ser reutilizada.
5. `sem_post(&barr->turnstile)`: Caso o processo que acabou de chegar seja o último esperado, libera todos os processos que estavam dormindo na barreira.
6. `sem_wait(&barr->turnstile)`: Caso ainda tenha processos para chegar, o processo que chegou dorme com os outros.
7. `sem_post(&barr->mutex)`: Em todo o caso, antes de sair destrava o mutex para liberar região crítica.

A barreira garante que nenhum processo passe antes dos outros, que todos os processos avancem juntos e que ela possa ser reutilizada posteriormente.

3.2. Implementação da Fila FIFO

```
void init_fifoQ(fifoq_t *F, int n_procs)
{
    // Mutex multiprocesso
    sem_init(&F->mutex_interno, 1, 1);
    F->head = 0;
    F->tail = 0;
    F->recurso_ocupado = 0;

    // Semáforos começam bloqueados
    for (int i = 0; i < n_procs; i++)
        sem_init(&F->sem_privado[i], 1, 0);
}

void inicia_uso(int pid_logico, int recurso, fifoq_t *F)
{
    // Trava mutex
    sem_wait(&F->mutex_interno);

    // Se não ocupado e fila vazia o processo avança
    if (F->recurso_ocupado == 0 && F->head == F->tail)
    {
        F->recurso_ocupado = 1;
        sem_post(&F->mutex_interno);
        return;
    }

    // Se ocupado processo entra na fila
    F->fila[F->tail] = pid_logico;
    F->tail = (F->tail + 1) % MAX_PROCS;

    sem_post(&F->mutex_interno);

    // Bloqueia seu próprio semáforo privado
    sem_wait(&F->sem_privado[pid_logico]);
}

void termina_uso(int recurso, fifoq_t *F)
{
    // Trava mutex
    sem_wait(&F->mutex_interno);

    // Se tem processo na fila
    if (F->head != F->tail)
    {
        int prox_processo = F->fila[F->head];
```

```

        F->head = (F->head + 1) % MAX_PROCS;
        sem_post(&F->sem_privado[prox_processo]); // Acorda o próximo
    }
    else
        // Sem não tem ninguém esperando libera o recurso
        F->recurso_ocupado = 0;

    sem_post(&F->mutex_interno);
}

```

3.2.1. Descrição

- void init_fifoQ(fifoq_t *F, int n_procs): Inicializa a fila, configurando todos campos da estrutura.
- void inicia_uso(int pid_logico, int recurso, fifoq_t *F): Entrada na região crítica e uso do recurso.
- void termina_uso(int recurso, fifoq_t *F) : Saída da região crítica e liberação do recurso.

3.2.2. Funcionamento

Na função `inicia_uso`:

1. `sem_wait(&F->mutex_interno)`: Protege a estrutura da fila na região crítica.
2. `if (F->recurso_ocupado == 0 && F->head == F->tail)`: Verifica se tem recurso livre e se a fila está vazia.
3. `F->recurso_ocupado = 1`: Se a fila está vazia e tem recurso, acesso imediato e marca como ocupado.
4. `F->fila[F->tail] = pid_logico`: Se a fila estiver ocupada ou não tiver recurso, insere o processo na fila e `F->tail = (F->tail + 1) % MAX_PROCS`; implementa a fila circular.
5. `sem_wait(&F->sem_privado[pid_logico])`: Cada processo espera seu próprio semáforo e nenhum processo é acordado erroneamente.
6. `sem_post(&F->mutex_interno)`: Destrava o mutex para liberar a região crítica antes do retorno.

Na função `termina_uso`:

1. `sem_wait(&F->mutex_interno)`: Protege a estrutura da fila na região crítica.
2. `if (F->head != F->tail)`: Verifica se tem algum processo esperando
3. `int prox_processo = F->fila[F->head]`: Se tem, seleciona o próximo e obtém o primeiro da fila.
4. `F->head = (F->head + 1) % MAX_PROCS`: Remove esse processo da fila

5. `F->recurso_ocupado = 0;` Se não tem ninguém esperando o recurso volta a ficar livre.
6. `sem_post(&F->mutex_interno);` Destrava o mutex para liberar a região crítica antes do retorno.

3.3. Funcionamento do Programa Principal

1. Criação da Memória Compartilhada
`shared_data_t *shm = mmap(NULL, sizeof(shared_data_t), PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);`
Isso permite que todos os processos acessem a barreira, fila e semáforos compartilhados. Sem isso, cada processo teria sua própria cópia das estruturas.
2. Inicialização das Estruturas: `init_barr(&shm->barreira, NUM_PROCS)` e `init_fifoQ(&shm->fila, NUM_PROCS)`.
3. Criação dos Processos: Utiliza `fork()` e `NUM_PROCS = 10` e cria 10 processos filhos.
4. Chegada na Primeira Barreira: Todos os processos precisam chegar em `process_barrier(&shm->barreira);` antes de continuar.
5. Simulação de Uso de Recurso: Cada processo executa `for (int uso = 0; uso < 3; uso++)` e passa por: Prólogo, Uso Exclusivo e Epílogo.
 - (a) Prólogo: Simula trabalho antes de solicitar o recurso. Ocorre a solicitação e a entrada na fila com `inicia_uso(...)`.
 - (b) Uso Exclusivo: Simula utilização do recurso compartilhado. Ocorre a liberação e entrega o recurso para o próximo da fila com `termina_uso(...)`
 - (c) Epílogo: Simula processamento após o uso.
6. Chegada na Segunda Barreira: Após todos os processos completarem seu três usos, todos devem sincronizar em `process_barrier(&shm->barreira);`.
7. Encerramento: O processo pai utiliza `wait()` para aguardar todos os filhos e traduz o PID real para o número lógico correspondente utilizando `tabela_pids[]` para mapear.

3.4. Resultados e Exemplo de Saída

3.4.1. Resultados Obtidos

Durante o desenvolvimento e testes do trabalho, foi possível verificar:

- Funcionamento correto da Barreira.
- Sincronização correta entre os processos.
- Garantia da política FIFO no acesso aos recursos.
- Ausência de `busy_wait` (espera ocupada).

3.4.2. Exemplo de Saída

Para esta saída, o exemplo foi gerado com apenas 3 processos para melhor visualização no relatório. Vale ressaltar que o código está apto a operar com um maior número de processos.

```

--Processo: 0 chegando na barreira
--Processo: 1 chegando na barreira
--Processo: 2 chegando na barreira
**Processo: 2 saindo da barreira
**Processo: 0 saindo da barreira
Processo: 2 Prologo: 0 de 1 segundos
Processo: 0 Prologo: 0 de 1 segundos
**Processo: 1 saindo da barreira
Processo: 1 Prologo: 0 de 2 segundos
Processo: 2 USO: 0 por 0 segundos
Processo: 2 Epilogo: 0 de 0 segundos
Processo: 0 USO: 0 por 3 segundos
Processo: 2 Prologo: 1 de 1 segundos
Processo: 0 Epilogo: 0 de 0 segundos
Processo: 1 USO: 0 por 3 segundos
Processo: 0 Prologo: 1 de 0 segundos
Processo: 1 Epilogo: 0 de 3 segundos
Processo: 2 USO: 1 por 2 segundos
Processo: 2 Epilogo: 1 de 3 segundos
Processo: 0 USO: 1 por 0 segundos
Processo: 0 Epilogo: 1 de 2 segundos
Processo: 1 Prologo: 1 de 1 segundos
Processo: 1 USO: 1 por 3 segundos
Processo: 0 Prologo: 2 de 3 segundos
Processo: 2 Prologo: 2 de 1 segundos
Processo: 1 Epilogo: 1 de 0 segundos
Processo: 2 USO: 2 por 2 segundos
Processo: 1 Prologo: 2 de 1 segundos
Processo: 2 Epilogo: 2 de 2 segundos
Processo: 0 USO: 2 por 3 segundos
--Processo: 2 chegando novamente na barreira
Processo: 0 Epilogo: 2 de 0 segundos
Processo: 1 USO: 2 por 2 segundos
--Processo: 0 chegando novamente na barreira
Processo: 1 Epilogo: 2 de 2 segundos
--Processo: 1 chegando novamente na barreira
++Processo: 1 saindo da barreira novamente
++Processo: 0 saindo da barreira novamente
++Processo: 2 saindo da barreira novamente
+++ Filho de número lógico 0 e pid 2187 terminou!
+++ Filho de número lógico 1 e pid 2188 terminou!
+++ Filho de número lógico 2 e pid 2189 terminou!

```

4. Decisões de Projeto

- Uso de Semáforos Privados: Ao invés de um unico semáforo para todos os processos, foi utilizado um semáforo individual, pois, acorda exatamente o próximo processo, evita competição desnecessária e garante a política FIFO.

- Uso de Fila Circular: Escolhida porque a inserção $O(1)$ e remoção $O(1)$ também, sem a movimentação de elemento.
- Reutilização da Barreira: A variável `count = 0` é reinicializada após a última chegada, isso permite que a mesma barreira seja utilizada múltiplas vezes durante a execução.
- Uso de `mmap`: Utilizada a chamada de sistema `mmap()` com as flags `MAP_SHARED` e `MAP_ANONYMOUS` para criar uma região de memória compartilhada entre o processo pai e os processos filhos. Essa abordagem permite o compartilhamento direto das estruturas de sincronização sem a necessidade de criar explicitamente segmentos de memória compartilhada através do uso de `shmget/shmat`, simplificando a implementação e mantendo o mesmo comportamento funcional.

5. Conclusão

O projeto implementa corretamente duas primitivas clássicas de sincronização entre processos:

- Barreira reutilizável, responsável por sincronizar grupos de processos em pontos específicos da execução.
- Fila FIFO para exclusão mútua, responsável por garantir acesso exclusivo e justo a um recurso compartilhado.

A solução utiliza memória compartilhada, semáforos POSIX e filas circulares, assegurando sincronização correta, ausência de condições de corrida e respeito à ordem de chegada dos processos. O resultado é um sistema robusto, reutilizável e compatível com os princípios clássicos de concorrência estudados em Sistemas Operacionais.